

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

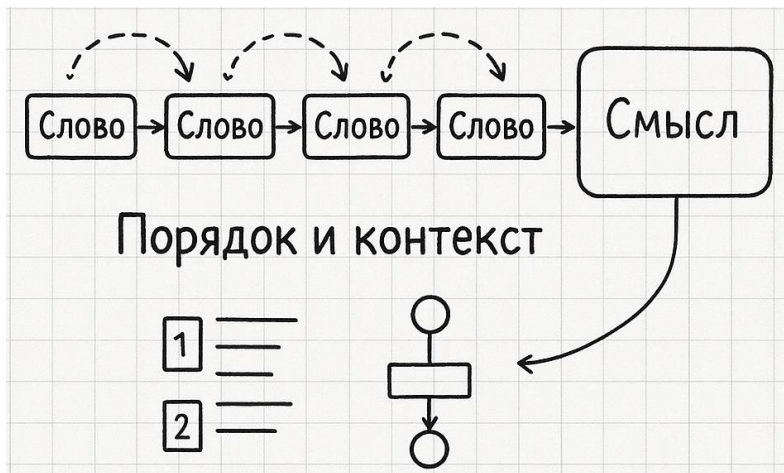
Анализ больших текстовых данных и текстовый поиск

Лекция 10

17 ноября 2025

Рекуррентные нейронные сети

Когда мы работаем с текстом, каждое слово зависит от предыдущих. Смысл фразы формируется постепенно, и модель должна помнить, что было раньше. Присутствует **временная компонента**. Практически все ключевые задачи NLP требуют учёта порядка и контекста.



Рекуррентные нейронные сети

NLP-задачи требующие памяти:

- **Классификация текста.** Чтобы понять тон или тему предложения, нужно учитывать весь контекст, а не отдельные слова
- **Language Modeling / автодополнение.** Следующее слово зависит от того, что уже написано
- **Named Entity Recognition (NER).** Чтобы решить, является ли слово частью сущности, важно, что стояло до него
- **Machine Translation.** Перевод предложения невозможен без понимания структуры всей фразы
- **Диалоговые системы.** Нужно отслеживать историю разговора, а не отдельные реплики.

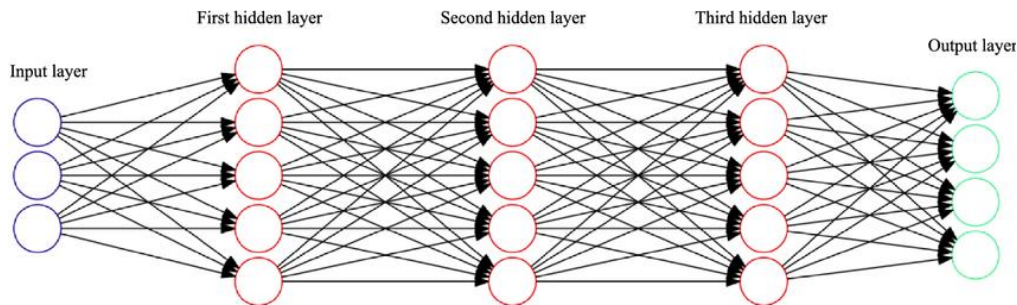
Рекуррентные нейронные сети

Трудности у полносвязных сетей

Полносвязный слой воспринимает вход как фиксированный вектор. Он не умеет:

- обрабатывать переменную длину текста
- помнить предыдущие элементы последовательности
- адаптироваться к структуре фразы

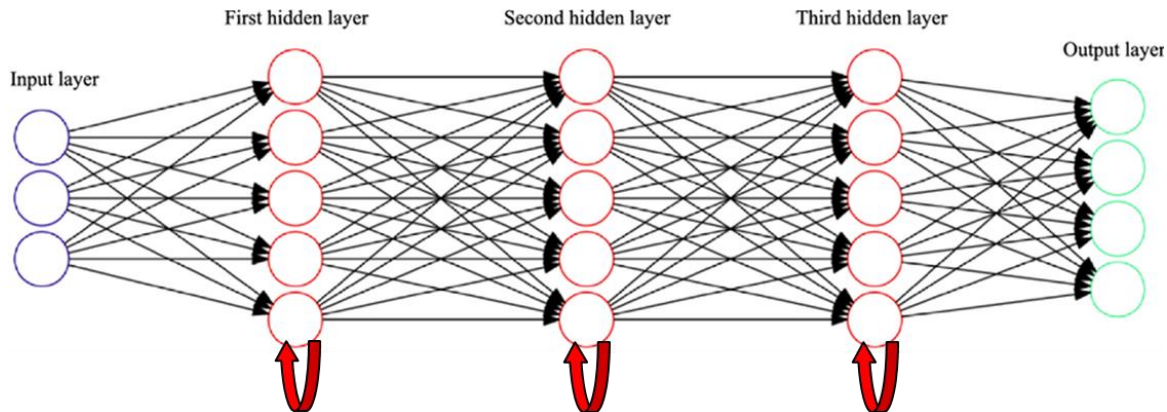
Если подать на вход длинный текст, его сначала нужно свести к фиксированной размерности, что приводит к потере порядка и контекста



Рекуррентные нейронные сети

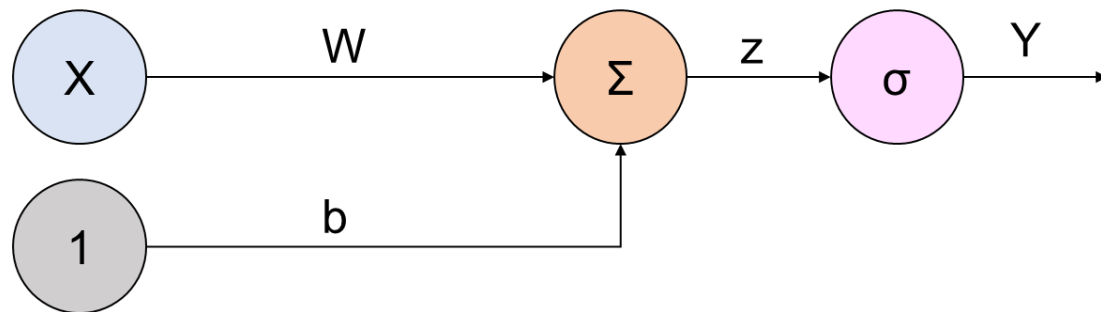
Идея рекуррентности

- Текст поступает по одному элементу
- Значение текущего слова зависит от того, что было раньше
- Модели должны уметь накапливать информацию в процессе чтения



Рекуррентные нейронные сети

Классический полносвязный слой



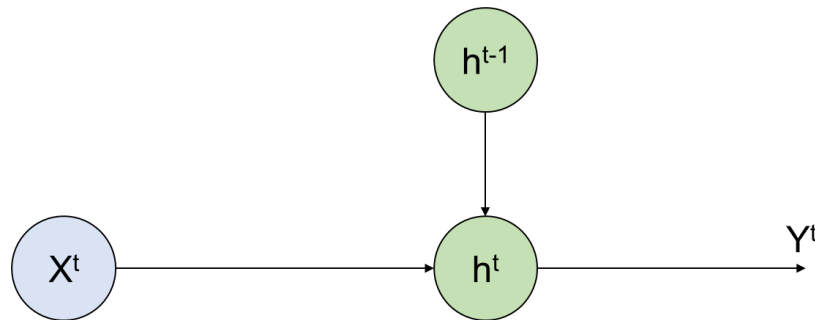
$$Y = \sigma(WX + b)$$

Обучаемые параметры: W , b

Рекуррентные нейронные сети

Рекуррентный слой

h^t — это вектор **скрытого состояния** (hidden state) рекуррентной нейронной сети в момент времени t



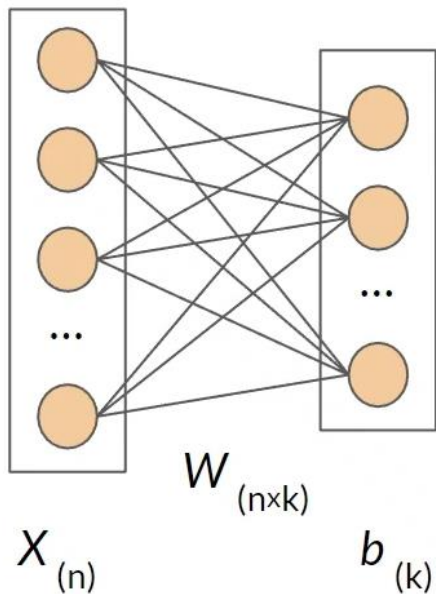
$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

$$y = \sigma(Vh^t + b_y)$$

Обучаемые параметры: W, U, V, b_h, b_y

Рекуррентные нейронные сети

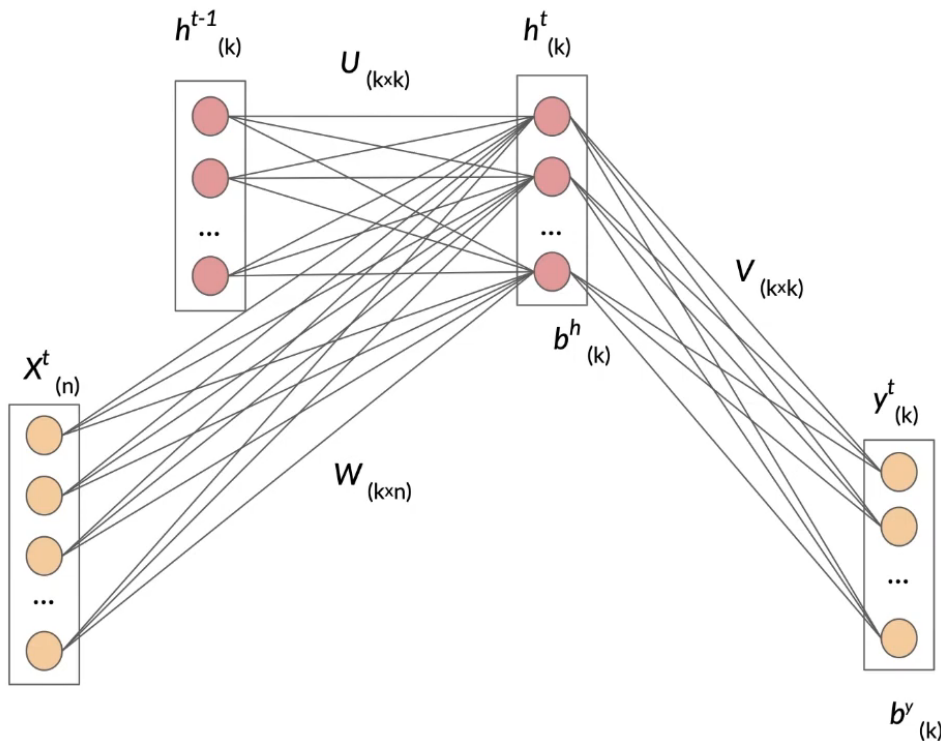
Если отобразить все веса, то полносвязный слой:



$$Y = \sigma(WX + b)$$

Рекуррентные нейронные сети

Тогда рекуррентный слой:



$$h^t = \sigma (W X^t + U h^{t-1} + b_h)$$

$$y = \sigma (V h^t + b_y)$$

Рекуррентные нейронные сети

Пример

a cat is sitting
on the mat

h^0_1

h^0_2

...

h^0_n

a $\begin{pmatrix} 0.45 \\ -1.34 \\ 2.34 \\ \dots \\ -0.45 \end{pmatrix}$

$\times^1$

→

h^1_1

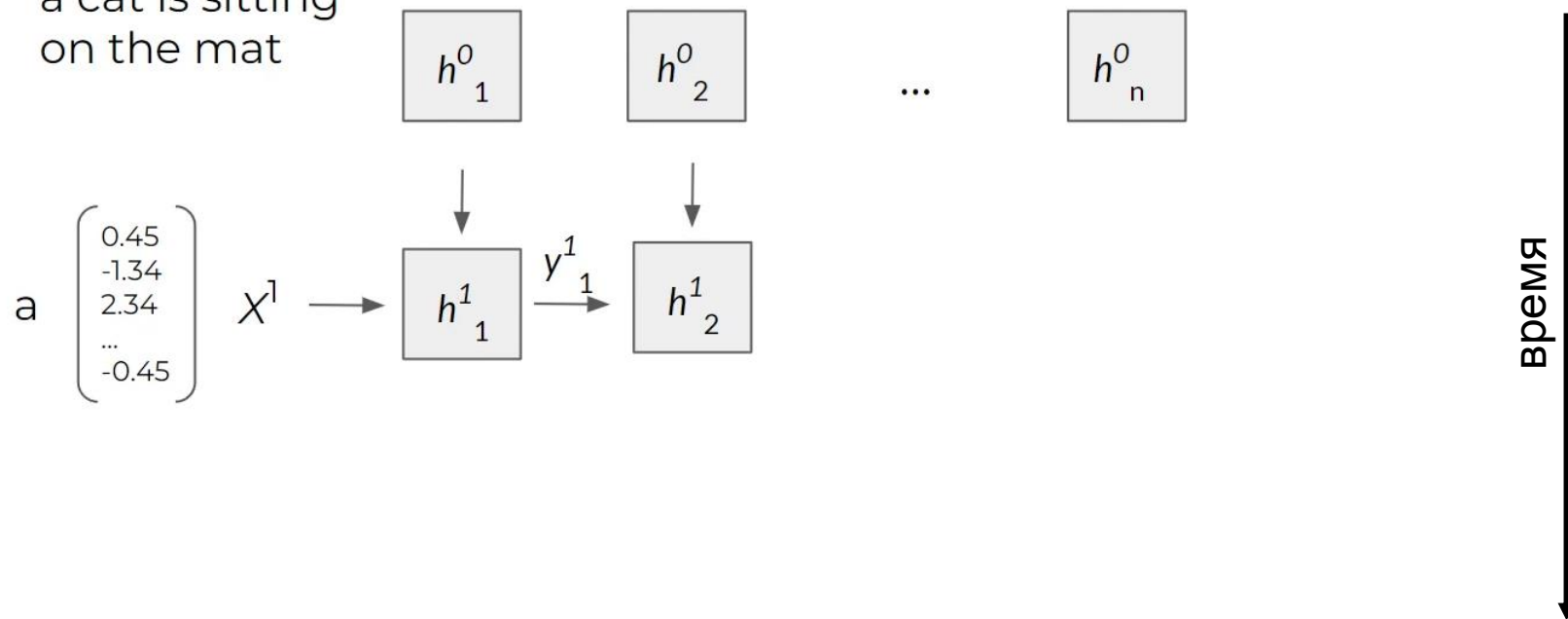
время



Рекуррентные нейронные сети

Пример

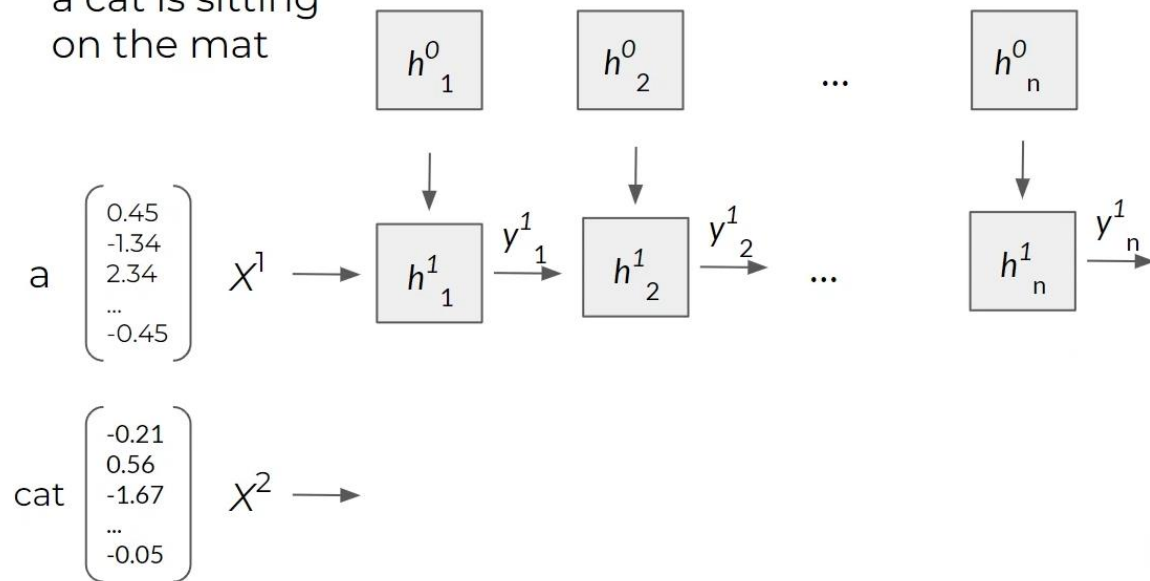
a cat is sitting
on the mat



Рекуррентные нейронные сети

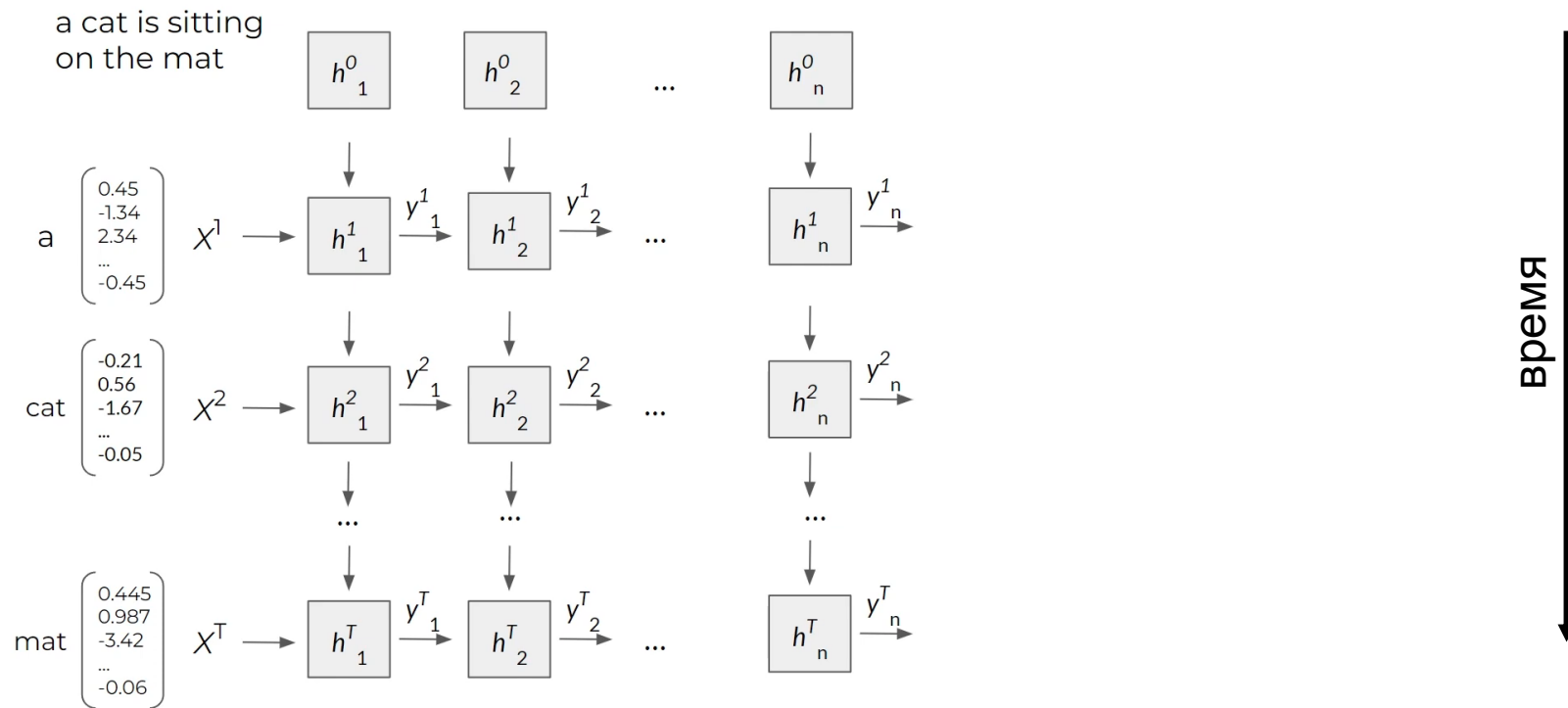
Пример

a cat is sitting
on the mat



Рекуррентные нейронные сети

Пример

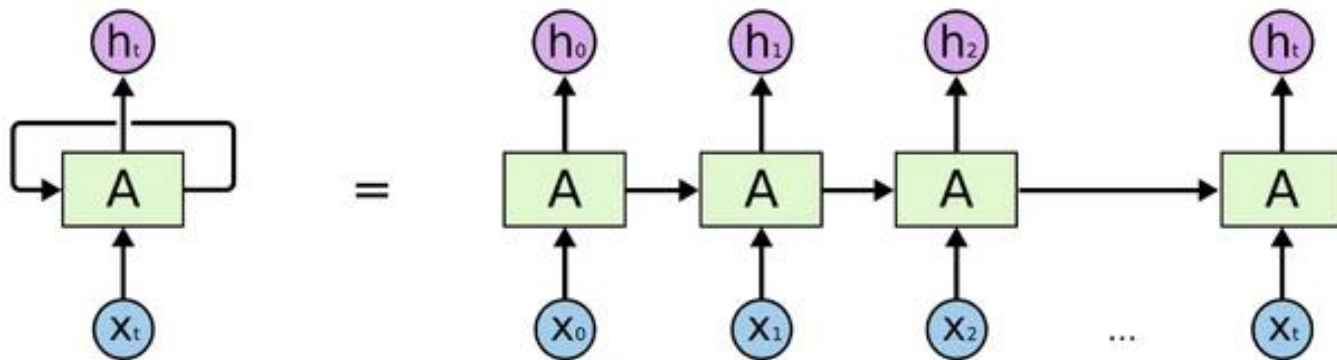


Рекуррентные нейронные сети

Обучение RNN

$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

$$y = \sigma(Vh^t + b_y)$$



Градиент функции потерь по W :

$$\frac{dL}{dW} = \frac{\partial L}{\partial h^T} \frac{\partial h^T}{\partial W} + \frac{\partial L}{\partial h^T} \frac{\partial h^T}{\partial h^{T-1}} \frac{\partial h^{T-1}}{\partial W} + \dots + \frac{\partial L}{\partial h^T} \frac{\partial h^T}{\partial h^{T-1}} \frac{\partial h^{T-1}}{\partial h^{T-2}} \dots \frac{\partial h^1}{\partial W}$$

Рекуррентные нейронные сети

Обучение RNN

$$\frac{dL}{dW} = \frac{\partial L}{\partial h^T} \frac{\partial h^T}{\partial W} + \frac{\partial L}{\partial h^T} \frac{\partial h^T}{\partial h^{T-1}} \frac{\partial h^{T-1}}{\partial W} + \dots + \frac{\partial L}{\partial h^T} \frac{\partial h^T}{\partial h^{T-1}} \frac{\partial h^{T-1}}{\partial h^{T-2}} \dots \frac{\partial h^1}{\partial W}$$

Элемент $\frac{\partial h^t}{\partial h^{t-1}}$ повторяется много раз

$$h^t = \sigma(Wx^t + Uh^{t-1} + b_h) \text{ , тогда } \frac{\partial h^t}{\partial h^{t-1}} = U \cdot \sigma'(\cdot)$$

$\sigma'(\cdot)$ — производная σ , вычисленная в том значении, которое стоит внутри активации для данного шага

$$\text{Для цепочки длиной } k: \frac{\partial h^T}{\partial h^{T-k}} = (U \cdot \sigma'(\cdot))^k$$

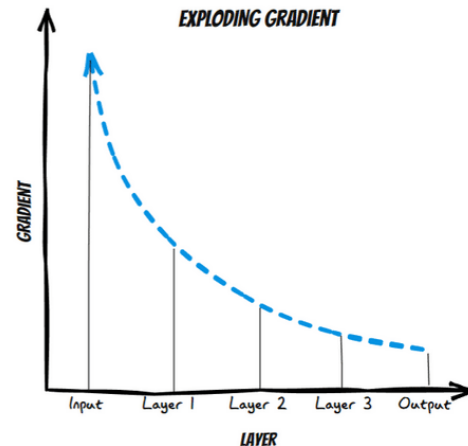
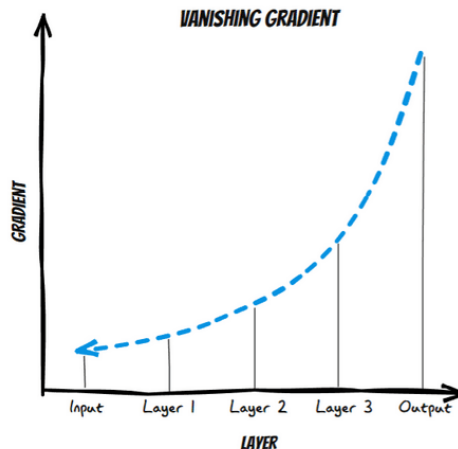
Рекуррентные нейронные сети

Обучение RNN

$$\frac{\partial h^T}{\partial h^{T-k}} = (U \cdot \sigma'(\cdot))^k$$

С ростом длины цепочки (k) это:

- или стремительно растёт
- или стремительно убывает



Рекуррентные нейронные сети

Функция потерь

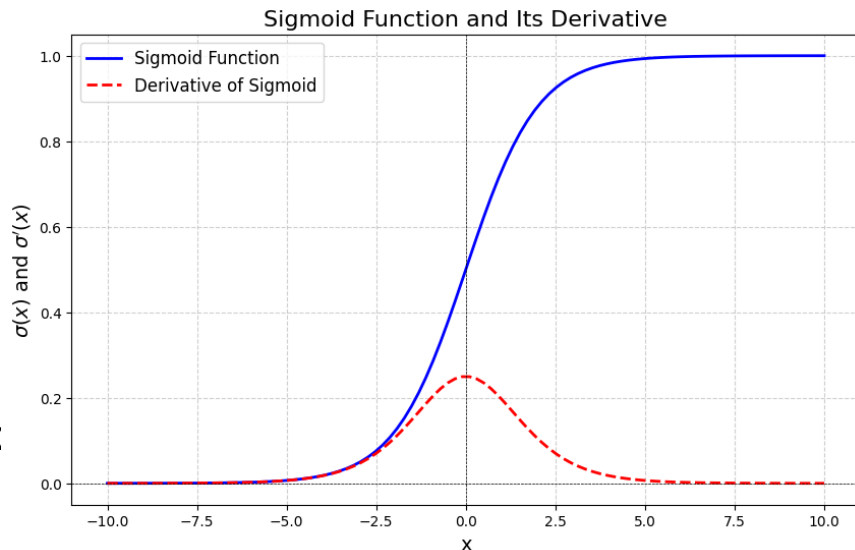
$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

Сигмоида приводит к сильному затуханию градиентов в RNN:

- маленькая производная даже в центре
- почти нулевая производная на больших значениях

Поэтому сигмоида в RNN не используется

$$\frac{\partial h^T}{\partial h^{T-k}} = (U \cdot \sigma'(\cdot))^k$$



Рекуррентные нейронные сети

Функция потерь

- Если значение внутри активации **положительное**:

$$\frac{\partial h^t}{\partial h^{t-1}} = U \cdot 1$$

При спектральном радиусе матрицы $U > 1$ градиент растёт экспоненциально

- Если значение внутри активации **отрицательное**:

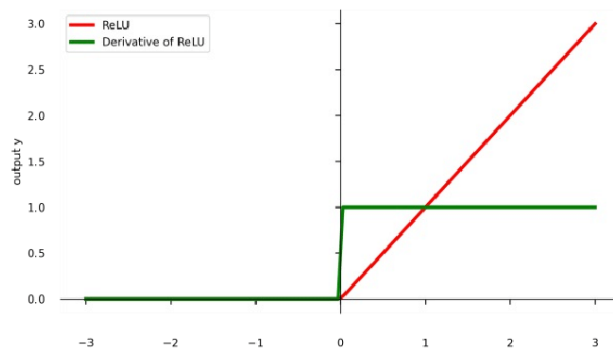
$$\text{ReLU}'(x) = 0$$

Тогда градиент перестаёт проходить назад и нейрон «умирает»

ReLU в RNN тоже не используется

$$\frac{\partial h^T}{\partial h^{T-k}} = (U \cdot \sigma'(\cdot))^k$$

$$\text{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$



Рекуррентные нейронные сети

Функция потерь

Гиперболический тангенс

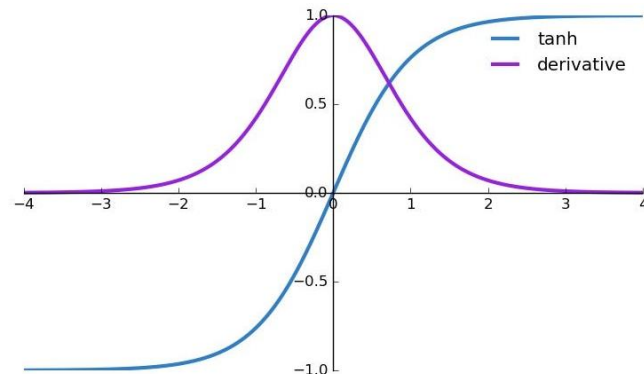
- слабее подвержен затуханию, чем сигмоида
- не ведёт себя так жёстко, как ReLU (без «мертвых» нейронов и резких скачков градиента)
- даёт простые и быстрые вычисления

Поэтому классическая RNN почти всегда использует
tanh в скрытом слое

$$\frac{\partial h^T}{\partial h^{T-k}} = (U \cdot \sigma'(\cdot))^k$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\tanh'(x) = 1 - \tanh^2(x)$$



Рекуррентные нейронные сети

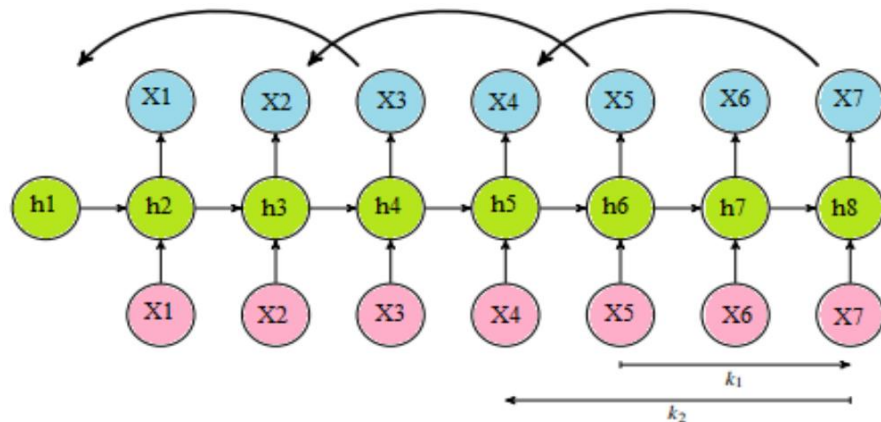
Truncated Backpropagation Through Time (обрезанное распространение ошибки во времени)

При обучении RNN развёртка по времени может получаться очень длинной: предложение бывает из десятков или сотен токенов. Если прогонять градиент через всю такую цепочку, обучение становится нестабильным и слишком тяжёлым по памяти. Поэтому на практике развёртку ограничивают фиксированным окном — модель пропускает градиент только через последние несколько шагов. Она всё ещё обрабатывает всю последовательность вперёд, но обратное распространение идёт только по выбранному фрагменту. Такой подход уменьшает риск взрыва или затухания градиентов, ускоряет обучение и обычно обозначается как **truncated BPTT**.

Рекуррентные нейронные сети

Truncated Backpropagation Through Time (обрезанное распространение ошибки во времени)

- RNN не нужно помнить всю последовательность при обучении
- Градиенты на далёких шагах вносят слишком мало пользы
- Сокращается время обучения
- Снижается риск нестабильности



Рекуррентные нейронные сети

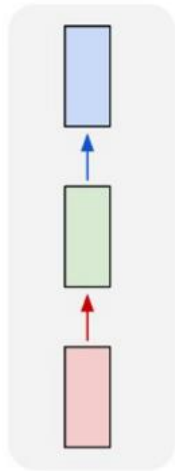
Gradient Clipping (ограничение градиента)

- При взрыве градиента обновления параметров становятся слишком резкими: веса скачут на большие значения, и обучение быстро выходит из-под контроля
- Clip вводит ограничение на норму градиента — если она превышает заданный порог, градиент масштабируется так, чтобы его величина стала безопасной
- Такой приём удерживает процесс обучения в стабильном режиме даже на длинных последовательностях, где взрыв градиентов встречается особенно часто
- Ограничение градиента выполняется автоматически на каждом шаге оптимизации, перед обновлением весов

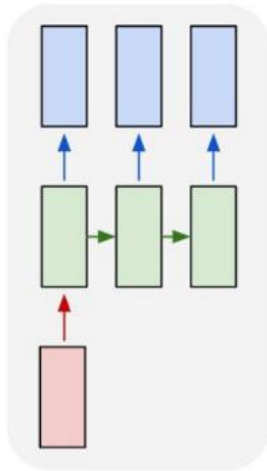
Рекуррентные нейронные сети

Типы RNN

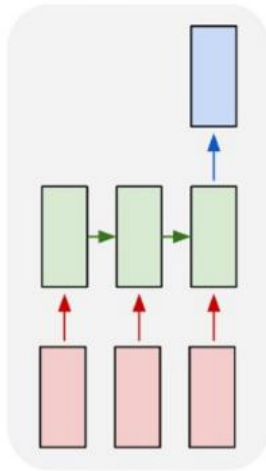
one to one



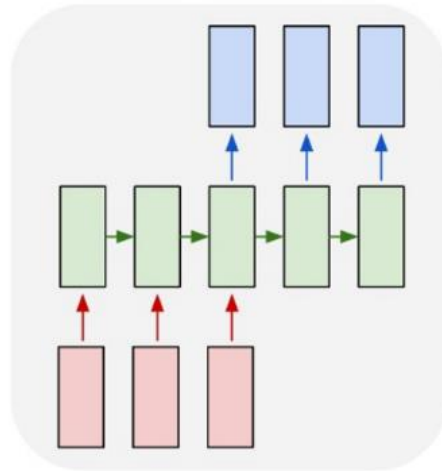
one to many



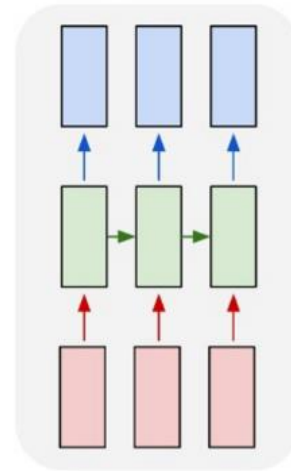
many to one



many to many



many to many

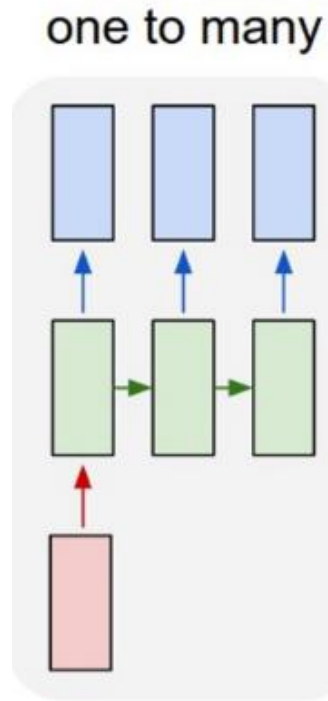


Рекуррентные нейронные сети

Типы RNN

One-to-many

- Один вход и последовательность выходов
- Пример: генерация текста по одному стартовому токену или условию
- Сеть получает начальное состояние и генерирует токены один за другим



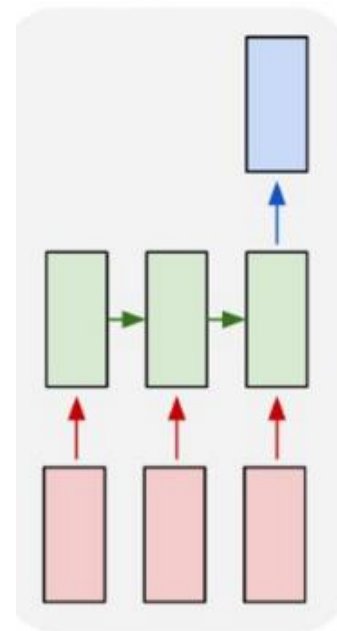
Рекуррентные нейронные сети

Типы RNN

Many-to-one

- Последовательность входов и один выход
- Основная схема для классификации текста
- Модель читает все токены по очереди, обновляя скрытое состояние
- Финальное состояние h^T обобщает всю информацию, накопленную по мере чтения
- Его подают в классификатор (часто линейный слой + softmax)

many to one



Рекуррентные нейронные сети

Типы RNN

Many-to-many

1. Асинхронный many-to-many

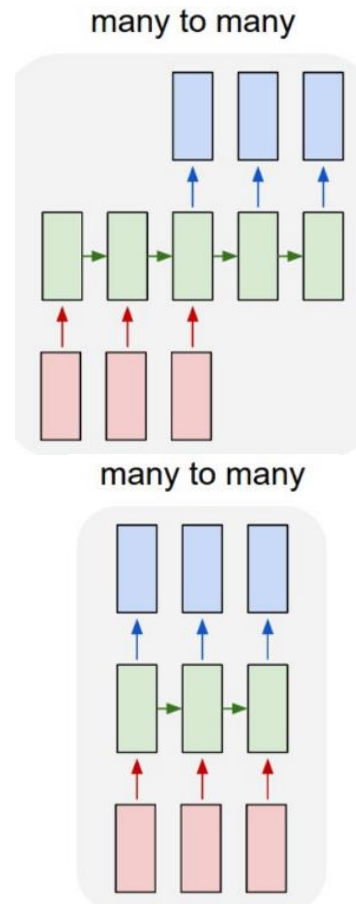
Сеть сначала полностью читает входную последовательность и только после этого начинает генерировать выход

Так работает машинный перевод: encoder читает весь текст, decoder затем выдаёт перевод

2. Синхронный many-to-many

Выход создаётся одновременно с входом: на каждый входной токен сразу выдаётся метка.

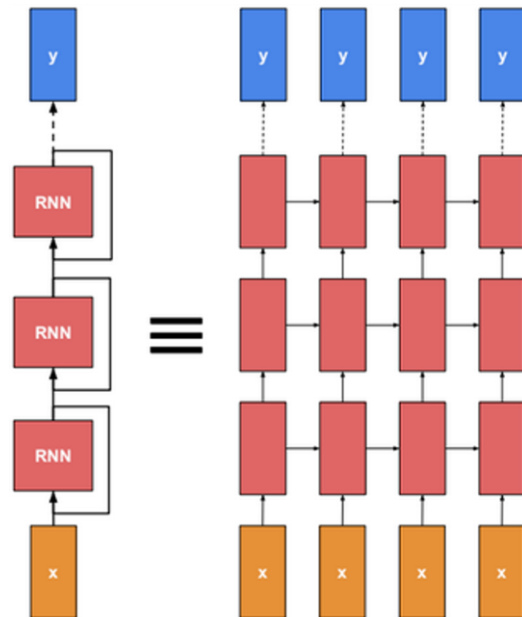
Так работают POS-теггинг, NER и другие задачи разметки



Рекуррентные нейронные сети

Многослойные RNN

Многослойные RNN позволяют модели читать текст не один раз, а проходить его через несколько уровней обработки. На каждом следующем слое сеть получает уже не сырые эмбединги, а более абстрактные представления, сформированные нижним уровнем. Благодаря этому модель улавливает более сложные зависимости и постепенно строит всё более высокоуровневое описание последовательности.

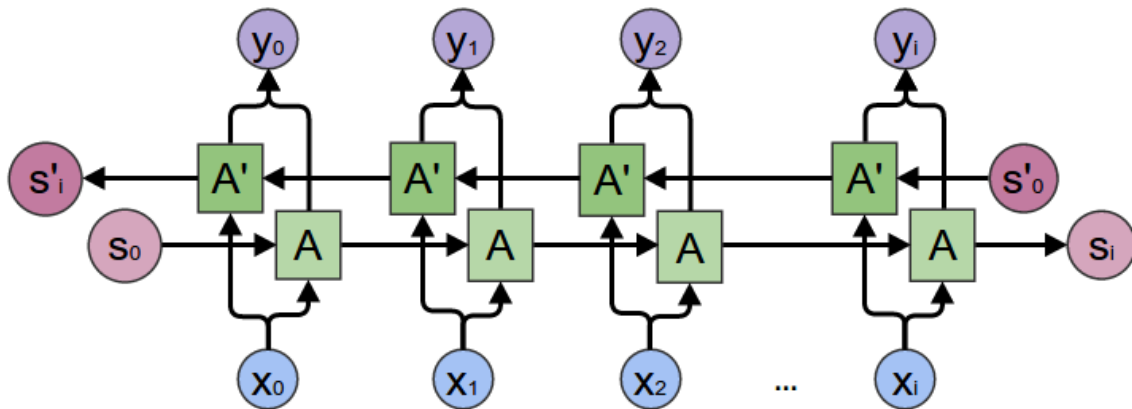


Рекуррентные нейронные сети

Bidirectional (двунаправленные) RNN

Идея: вместо одного потока используются два — прямой и обратный. Один читает последовательность слева направо, другой — справа налево.

Такая архитектура позволяет учитывать контекст по обе стороны от текущего токена. Это особенно важно там, где значение слова определяется соседями: задачи разметки, NER, POS



Рекуррентные нейронные сети

Классические рекуррентные сети испытывают несколько фундаментальных ограничений, которые мешают им эффективно работать на реальных последовательностях.

Во-первых, из-за использования Backpropagation Through Time возникает серьёзная проблема затухающего градиента: при обратном проходе вклад далёких шагов становится настолько маленьким, что обновление весов практически не происходит. В результате обучение длительных зависимостей замедляется или вовсе прекращается

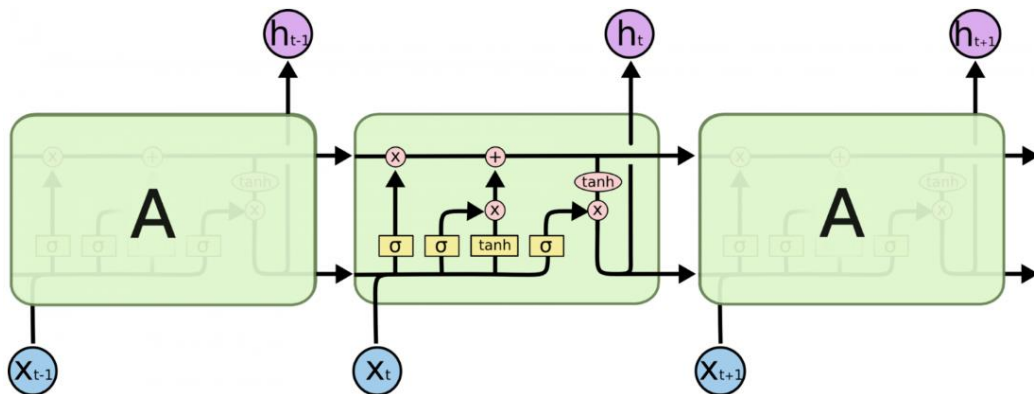
Во-вторых, при последовательном обновлении скрытого состояния информация из начала текста постепенно вытесняется новыми токенами. На длинных последовательностях ранний контекст просто утрачивается, и модель работает лишь с последними фрагментами ввода. Это и называют эффектом короткой памяти

Из-за этих ограничений стандартные RNN способны давать приемлемое качество только на сравнительно коротких текстах, где зависимостей немного. Для более сложных последовательностей понадобились усовершенствованные варианты RNN-ячеек, специально разработанные для сохранения долгосрочной информации

LSTM (Long Short-term Memory)

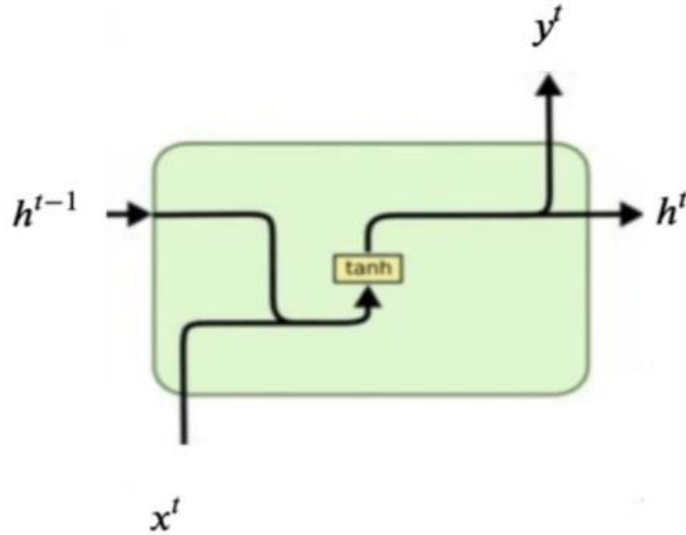
Ключевая идея LSTM

- Вводится отдельная память, называемая cell state (долгосрочная память)
- Память может передаваться между шагами почти без изменений
- Механизм ворот управляет тем, что сохранять, а что забывать



LSTM (Long Short-term Memory)

Схема ячейки классической RNN



$$h^t = \tanh(WX^t + Uh^{t-1} + b_h)$$

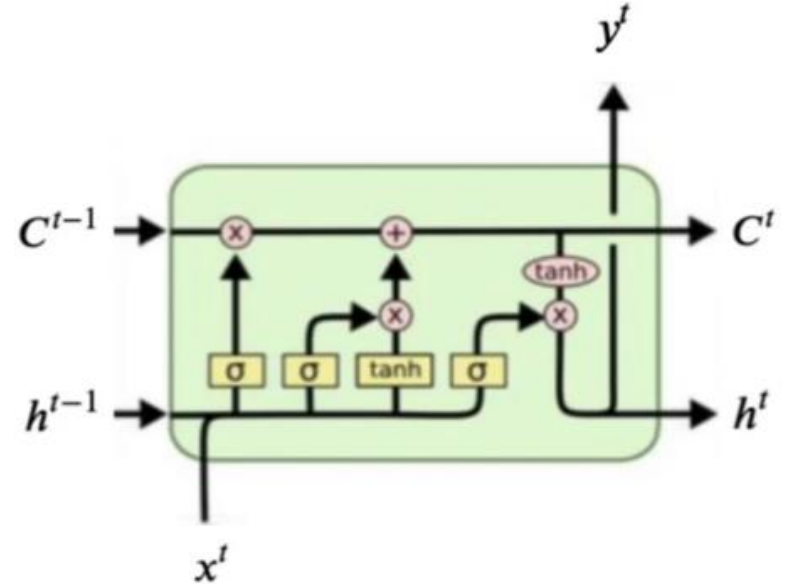
$$y^t = \sigma(W_y h^t + b_y)$$

LSTM (Long Short-term Memory)

Схема ячейки LSTM

- C^t — долгосрочная память
- h^t — краткосрочная память

В LSTM состояние C^t отвечает за долгосрочную память, которая переносит важную информацию через всю последовательность, а h^t служит краткосрочным состоянием, участвующим в вычислениях текущего шага.



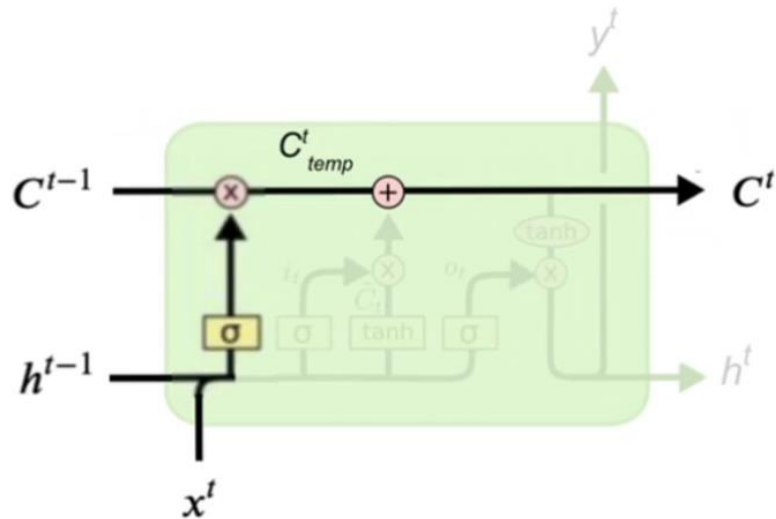
LSTM (Long Short-term Memory)

Forget gate (ворота забывания)

Ворота забывания вычисляют вектор f^t через сигмоиду, используя прошлое состояние h^{t-1} и текущий вход x^t . Этот вектор содержит значения от 0 до 1 и поэлементно умножается на C^{t-1} , определяя, какая часть старой долгосрочной памяти будет сохранена, а какая — стерта

$$f^t = \sigma(W_f [h^{t-1}, x^t] + b_f)$$

$$C_{\text{temp}}^t = f^t * C^{t-1}$$



LSTM (Long Short-term Memory)

Input gate (ворота входа)

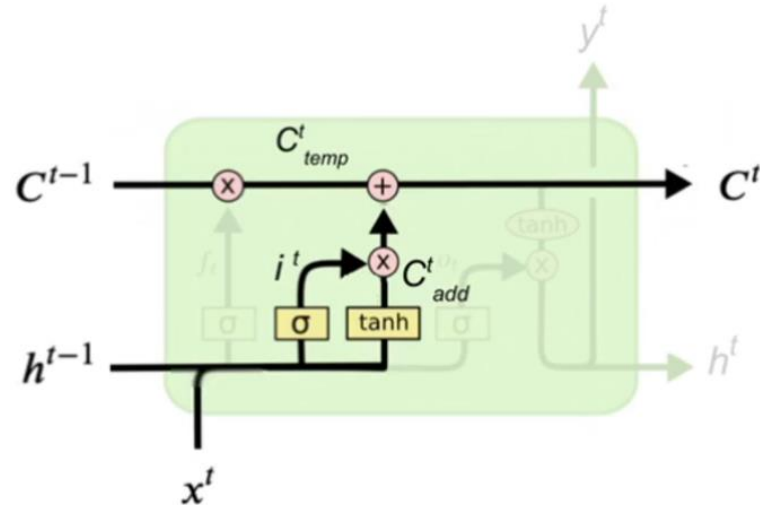
Ворота входа вычисляют, какая часть новой информации (C_{add}^t , полученной через **tanh**) действительно должна попасть в память

Сигмоида даёт вектор i^t от 0 до 1, который поэлементно регулирует, сколько этой новой информации добавляется в C^t

$$i^t = \sigma(W_i [h^{t-1}, x^t] + b_i)$$

$$C_{add}^t = \tanh(W_C [h^{t-1}, x^t] + b_C)$$

$$C^t = C_{temp}^t + i^t * C_{add}^t$$



LSTM (Long Short-term Memory)

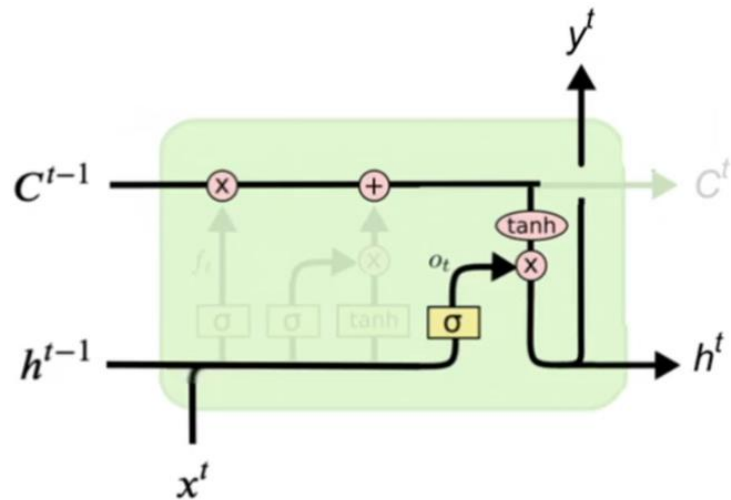
Обновление h^t и получение выхода y^t

Ворота выхода формируют вектор o^t — значения от 0 до 1, которые регулируют, какую часть обработанной памяти $\tanh(C^t)$ нужно передать в краткосрочное состояние h^t . После этого полученный вектор h^t подаётся на линейный слой с функцией активации, чтобы получить финальный выход ячейки y^t .

$$o^t = \sigma(W_o [h^{t-1}, x^t] + b_o)$$

$$h^t = o^t * \tanh(C^t)$$

$$y^t = \sigma(W_y h^t + b_y)$$



LSTM (Long Short-term Memory)

Полный цикл

LSTM заметно сложнее классической RNN, потому что внутри одной ячейки происходит гораздо больше вычислений. Если обычная RNN обновляет состояние с помощью одной простой формулы, то LSTM использует несколько отдельных ворот, каждое из которых имеет свои веса и выполняют собственные операции: одни решают, что забыть, другие — что добавить, третьи — что отдать наружу. Из-за этого количество параметров увеличивается в несколько раз, возрастает вычислительная нагрузка, и обучение становится тяжелее. Однако эта сложность окупается: LSTM гораздо лучше удерживает полезную информацию на длинных последовательностях и существенно стабильнее при обучении.

$$f^t = \sigma(W_f [h^{t-1}, x^t] + b_f)$$

$$i^t = \sigma(W_i [h^{t-1}, x^t] + b_i)$$

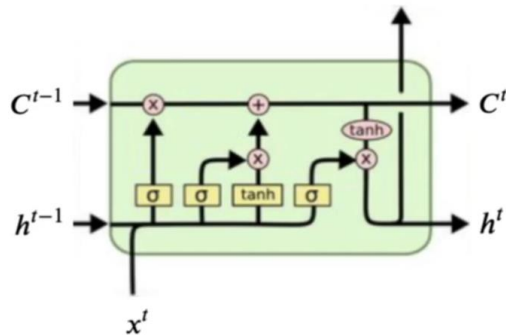
$$C_{add}^t = \tanh(W_C [h^{t-1}, x^t] + b_C)$$

$$C^t = f^t * C^{t-1} + i^t * C_{add}^t$$

$$o^t = \sigma(W_o [h^{t-1}, x^t] + b_o)$$

$$h^t = o^t * \tanh(C^t)$$

$$y^t = \sigma(W_y h^t + b_y)$$



Ограничения RNN

Проблемы с долгими зависимостями

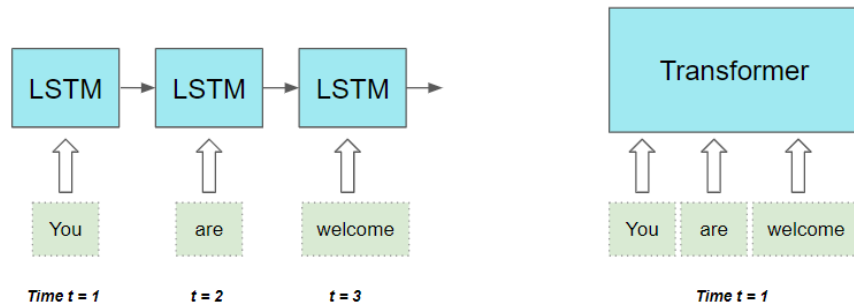
Даже с улучшенными ячейками вроде LSTM сети всё равно испытывают трудности с контекстами на сотни токенов — информация постепенно теряется

Отсутствие параллелизации

RNN обрабатывают последовательность строго по шагам, поэтому обучение и инференс сильно медленнее по сравнению с методами, которые можно распараллелить

Почему их вытеснили трансформеры

Трансформеры используют механизм внимания и читают всю последовательность сразу, без пошагового прохода. Они удерживают дальние зависимости, легко масштабируются и обучаются быстрее, что сделало их стандартом в NLP



Актуальность RNN

Компактные edge-модели

На устройствах с ограниченными ресурсами (Jetson Nano, микрокомпьютеры, embedded-системы) RNN часто выигрывают за счёт маленького числа параметров и низкого потребления памяти

Простые временные ряды

Задачи прогнозирования с небольшими паттернами, где нет экстремально длинных зависимостей, всё ещё решаются с помощью RNN

Лёгкие модели для микроконтроллеров

RNN достаточно компактны, чтобы работать даже на микроконтроллерах, где доступно всего несколько сотен килобайт памяти, поэтому их часто используют в небольших IoT-устройствах

